



BrightLearn
AI & DATA SKILLS

PRACTICAL ASSIGNMENT · 100
MARKS

Healthcare EDA Assignment

Explore a real-world-style healthcare admissions dataset with Python and pandas. Clean the data, engineer new features, answer twenty questions, and present your findings.

TOOL

Python 3 · pandas

DATASET

~55,500 rows · 15 columns

DELIVERABLE

One .ipynb notebook

QUESTIONS

Q1 – Q20 · 10 sections



00 Welcome & Objective

In this practical you will use Python and the pandas library to explore a real-world-style healthcare admissions dataset. Each row represents a single hospital admission. Your task is to understand the data, clean the obvious problems, engineer useful new columns, answer a set of analysis questions, visualise your findings, and present everything in a Jupyter notebook written for a non-technical reader — imagine a hospital manager.

DEFINITION

What is being assessed

You are not only marked on whether your code runs, but on whether you can **interpret the data and communicate what you found**. Code with no explanation earns very few marks; explanation with no supporting evidence earns even fewer. Aim for clear, honest, evidence-based analysis throughout.

★ THE TEN PARTS YOUR NOTEBOOK MUST CONTAIN

- **01 Data overview** — shape, columns, dtypes, a sample.
- **02 Data dictionary** — every column described in your own words.
- **03 Data-quality checks** — missing values, duplicates, unusual values.
- **04 Cleaned dataset** — fixes applied on a copy, with justification.
- **05 Feature engineering** — eight new columns, three of them built with `.loc`.
- **06 EDA analysis tables** — grouped and sorted answers to Q7 – Q18.
- **07 At least 5 charts** — each with a title and labelled axes.
- **08 Written insights** — five evidence-based findings.
- **09 Final executive summary** — roughly 150 – 200 words for a busy manager.
- **10 Recommendations** — three or four actionable suggestions.



01 The Dataset & Setup

The dataset is a CSV file with roughly 55,500 admission records across 15 columns. Each row is one hospital admission. You will document what each column means yourself in Section 2 — that is part of the exercise, so this brief does not define them for you.

THE FIFTEEN COLUMNS

- | | | |
|-------------------------|----------------------------|-----------------------------|
| • Name | • Age | • Gender |
| • Blood Type | • Medical Condition | • Date of Admission |
| • Doctor | • Hospital | • Insurance Provider |
| • Billing Amount | • Room Number | • Admission Type |
| • Discharge Date | • Medication | • Test Results |

★ BEFORE YOU START

- **Place the file correctly** — healthcare_dataset.csv must sit in the same folder as your notebook.
- **Work top to bottom** — each section builds on the previous one. Do not skip around.
- **Show your working** — print() or display every result so the marker can see it.
- **Write in plain English** — explanations go in markdown cells, not in code comments.
- **Confirm before submitting** — Kernel → Restart & Run All must complete cleanly from a fresh start.
- **Name the file correctly** —
Healthcare_EDA_<YourName>_<StudentNumber>.ipynb.



ACADEMIC INTEGRITY

This is **individual work**. You may discuss ideas with classmates, but the code and



the write-up must be your own. Submissions are checked.



02 Required New Columns

You must create all eight of the columns below in Section 5. The three band columns — Age_Group, Billing_Bucket, and Stay_Category — must be built using the .loc pattern shown later in this brief.

Column	What it should contain
Length_of_Stay	Days between admission and discharge
Admission_Year	Year of admission
Admission_Month	Month number of admission (1 – 12)
Admission_Month_Name	Month name of admission (e.g. January)
Admission_Day_Name	Weekday name of admission (e.g. Monday)
Age_Group	Age band — use .loc
Billing_Bucket	Billing-amount band — use .loc
Stay_Category	Short / Medium / Long stay — use .loc



03

Section 1 — Data Overview

DATA LOADING & FIRST
INSPECTION · 10 MARKS

Load the dataset, then get to know it. Three short questions. Show every result with `print()` or `display()` so the marker can see your output.

QUESTION Q1

How many rows and columns are in the dataset?

HINT

```
df.shape      # returns (rows, columns)
```

QUESTION Q2

What columns are available?

HINT

```
list(df.columns)  # wrap in list(...) to print cleanly
```

QUESTION Q3

What are the data types?

HINT

```
df.info()      # dtypes + non-null counts in one place
df.dtypes      # just the dtypes

# Also get a feel for the values:
df.sample(5)   # five random rows
df.describe()  # summary stats for numeric columns
```



★ **WHAT THE MARKER WANTS TO SEE**

Three pieces of evidence: the shape, the column list, and the dtypes. Plus a quick sample so you have seen the actual values before you start cleaning. Without this, every later step is guesswork.



04

Section 2 — Data Dictionary

Build a markdown table that describes every column in your own words and gives a realistic example value. This forces you to look at the data carefully — and it is what an analyst would hand to a manager who does not work with the data every day.

USE THIS TEMPLATE — FILL IT IN FOR ALL 15 COLUMNS

Column	Data type	What it means (your words)	Example value
Name	...	...	...
Age	...	...	...
Gender	...	...	...
Blood Type	...	...	...
Medical Condition	...	...	...
Date of Admission	...	...	...
Doctor	...	...	...
Hospital	...	...	...
Insurance Provider	...	...	...
Billing Amount	...	...	...
Room Number	...	...	...
Admission Type	...	...	...
Discharge Date	...	...	...
Medication	...	...	...
Test Results	...	...	...



05

Section 3 — Data-Quality Checks

DATA-QUALITY CHECKS ·
15 MARKS

Real data is rarely perfect. Investigate Q4 – Q6 – do not assume the data is clean just because it loaded without errors. At the end of this section write a short summary listing every problem you found.

QUESTION Q 4

Are there missing values?

HINT

```
df.isna().sum()      # count of missing values per column
```

QUESTION Q 5

Are there duplicate rows?

HINT

```
df.duplicated().sum()  # how many rows are exact copies?
```

QUESTION Q 6

Are there unusual numeric values?

HINT

```
df["Age"].min(), df["Age"].max()  
df["Billing Amount"].min(), df["Billing Amount"].max()
```

```
# Could a bill ever be negative? Count them:
```

```
(df["Billing Amount"] < 0).sum()
```

```
# Also glance at text columns – do names look consistently formatted?
```



```
df["Name"].head(10)
```

★ THE DATA-QUALITY SUMMARY

Write three to five sentences listing every problem you found and how serious each one is. "Six negative billing amounts (~0.01% of rows) — minor." "Patient names mixed case — cosmetic, easy fix." That style. The marker is looking for a clear, calm assessment — not a panic list.



06

Section 4 — Data Cleaning

DATA CLEANING & DATE
CONVERSION · 15 MARKS

Fix the problems you found. Work on a copy of the DataFrame (`df_clean = df.copy()`) so the original stays intact, and explain each decision in a markdown cell.

4.1 REMOVE DUPLICATE ROWS

PYTHON

```
print(df_clean.shape)          # before
df_clean = df_clean.drop_duplicates()
print(df_clean.shape)          # after – confirms the change
```

4.2 CONVERT DATES FROM TEXT TO DATETIME

PYTHON

```
df_clean["Date of Admission"] = pd.to_datetime(df_clean["Date of Admission"])
df_clean["Discharge Date"]    = pd.to_datetime(df_clean["Discharge Date"])

df_clean.dtypes               # check both columns are now datetime64[ns]
```

4.3 DEAL WITH BAD BILLING VALUES

Decide what to do with negative bills — remove the rows, set them to missing, or take the absolute value. Justify your choice in a markdown cell. There is no single right answer, but you must defend the trade-off.

HINT

```
# Three valid approaches – choose one and defend it:

# Option A – drop the rows entirely (loses data)
df_clean = df_clean[df_clean["Billing Amount"] >= 0]
```



```
# Option B – set them to NaN (keeps the other columns)
df_clean.loc[df_clean["Billing Amount"] < 0, "Billing Amount"] = pd.NA

# Option C – take the absolute value (assumes a sign error)
df_clean["Billing Amount"] = df_clean["Billing Amount"].abs()
```

4.4 TIDY THE PATIENT NAMES (RECOMMENDED)

P Y T H O N

```
df_clean["Name"] = df_clean["Name"].str.title()    # ALICE smith → Alice Smith
```

★ THE CLEANING NOTE

Write a short paragraph stating exactly what you changed and why — especially how you treated the bad billing values and what trade-off that involves. This is one of the few places in the assignment where there's no "correct" answer; the marks are for clarity of judgement.



07

Section 5 — Feature Engineering

FEATURE ENGINEERING USING
.loc · 15 MARKS

Create all eight required columns. The date-based ones are quick. For the three band columns you must use `.loc` to assign labels by condition — study the pattern below.

THE `.loc` PATTERN — STUDY THIS, APPLY TO EACH BAND

PYTHON

```
# 1) start with a default label for the whole column
df_clean["Example_Band"] = "unset"

# 2) overwrite by condition using .loc[row_condition, "column"] = value
df_clean.loc[df_clean["Age"] >= 65, "Example_Band"] = "Senior"
df_clean.loc[df_clean["Age"] < 65, "Example_Band"] = "Not senior"
```

★ WHY `.loc` AND NOT `df[df.Age>=65]["col"] = ...`

Chained indexing like `df[df.Age>=65]["col"] = ...` often updates a temporary copy and not the real DataFrame — it triggers the famous `SettingWithCopyWarning` and silently fails. `.loc[rows, column] = value` is the safe, recommended way to assign values to a subset of rows. Always use it.

THE DATE-BASED COLUMNS — ONE-LINERS

PYTHON

```
df_clean["Length_of_Stay"] = (df_clean["Discharge Date"]
                              - df_clean["Date of Admission"]).dt.days
df_clean["Admission_Year"] = df_clean["Date of Admission"].dt.year
df_clean["Admission_Month"] = df_clean["Date of Admission"].dt.month
df_clean["Admission_Month_Name"] = df_clean["Date of Admission"].dt.month_name()
df_clean["Admission_Day_Name"] = df_clean["Date of Admission"].dt.day_name()
```

SUGGESTED BANDS (ADJUST IF YOU CAN JUSTIFY IT)



Band column	Suggested cuts	Labels
Age_Group	0 – 17 · 18 – 35 · 36 – 50 · 51 – 65 · 66+	Child / Young adult / Adult / Senior / Elderly
Billing_Bucket	< 10k · 10k – 20k · 20k – 30k · 30k +	Low / Medium / High / Very High
Stay_Category	1 – 3 days · 4 – 7 days · 8+ days	Short / Medium / Long

★ **BEFORE YOU MOVE ON**

Display a few rows of the cleaned DataFrame showing your new columns so the marker can see them. `df_clean.head()` with a wide enough display width is enough.



08

Section 6 — Analysis & Groupby

GROUPBY ANALYSIS &
SORTING · 20 MARKS ·
LARGEST SECTION

Answer Q7 – Q18. For each one, produce a table (counts or grouped aggregates), sort it so the most important rows are on top, and add a one-line comment saying what it tells you. This is the largest section – 20 marks – so do not rush it.

★ THE TOOLS YOU WILL USE THE MOST

`value_counts()` for counting categories. `groupby(...).agg(...)` for grouped summaries. `sort_values(...)` to put the important rows on top. `pd.crosstab(...)` for two-way tables. Master these four and Section 6 is straightforward.

QUESTION Q 7

What are the most common medical conditions?

HINT

```
df_clean["Medical Condition"].value_counts()
```

QUESTION Q 8

Which admission type is most common?

HINT

```
df_clean["Admission Type"].value_counts()
```

QUESTION Q 9

Which insurance providers have the most patients?

**HINT**

```
df_clean["Insurance Provider"].value_counts()
```

QUESTION Q10

What is the age profile of patients?

HINT

```
df_clean["Age"].describe()           # min, max, mean, quartiles  
df_clean["Age_Group"].value_counts() # counts per band
```

QUESTION Q11

Which medical conditions are most common by age group?

HINT

```
pd.crosstab(df_clean["Age_Group"], df_clean["Medical Condition"])
```

QUESTION Q12

What is the total and average billing amount by medical condition?

HINT

```
(df_clean.groupby("Medical Condition")["Billing Amount"]  
    .agg(["sum", "mean", "count"])  
    .sort_values("sum", ascending=False))
```

QUESTION Q13

Which admission type has the highest average billing amount?

HINT

```
(df_clean.groupby("Admission Type")["Billing Amount"]  
    .mean())
```



```
.sort_values(ascending=False))
```

QUESTION Q14

What is the average length of stay by medical condition?

HINT

```
(df_clean.groupby("Medical Condition")["Length_of_Stay"]  
    .mean()  
    .sort_values(ascending=False))
```

QUESTION Q15

How do test results differ by medical condition?

HINT

```
pd.crosstab(df_clean["Medical Condition"], df_clean["Test Results"])  
  
# For percentages instead of counts:  
pd.crosstab(df_clean["Medical Condition"], df_clean["Test Results"],  
            normalize="index") * 100
```

QUESTION Q16

Which medications are most commonly prescribed for each condition?

HINT

```
df_clean.groupby("Medical Condition")["Medication"].value_counts()  
  
# Or – most common medication per condition in one line:  
df_clean.groupby("Medical Condition")["Medication"].agg(lambda s:  
s.mode().iloc[0])
```

QUESTION Q17



How have admissions changed over time?

H I N T

```
df_clean["Admission_Year"].value_counts().sort_index()  
df_clean["Admission_Month_Name"].value_counts()
```

Q U E S T I O N Q 1 8

Which hospitals and doctors have the most admissions?

H I N T

```
df_clean["Hospital"].value_counts().head(10)  
df_clean["Doctor"].value_counts().head(10)
```

Note what the size of those top counts tells you about the data –
is the data spread across many providers, or concentrated in a few?

★ THE ONE-LINE RULE

After every table, write one sentence in a markdown cell saying what the result tells you. "Diabetes is the most common condition, with about X admissions." Without these one-liners, you'll lose Section 8 marks too – because there will be no analytical thread for the marker to follow.



09

Section 7 — Visualisations

VISUALISATIONS · 10
MARKS

Produce at least five charts that support your analysis. Every chart must have a title and labelled axes. Use pandas built-in plotting plus matplotlib labels — fastest route from a DataFrame to a publication-ready chart.

★ FIVE SUGGESTED CHARTS — SWAP IN YOUR OWN IF YOU PREFER

- **Bar chart** — admissions by medical condition.
- **Bar chart** — patients by insurance provider (or by admission type).
- **Histogram** — age distribution.
- **Line or bar chart** — admissions over time (by year or month).
- **Bar chart** — average billing amount by medical condition.
- **Bonus** — box plot of billing by admission type, or stacked bar of test results by condition.

THE FASTEST CHART RECIPE**P Y T H O N**

```
import matplotlib.pyplot as plt

(df_clean["Medical Condition"]
 .value_counts()
 .plot(kind="bar"))

plt.title("Admissions by Medical Condition")
plt.xlabel("Medical Condition")
plt.ylabel("Number of Admissions")
plt.tight_layout()
plt.show()
```



 **A CHART WITHOUT
LABELS IS NOT A CHART**

Every chart needs three things: a clear title that states the message, a labelled X axis, and a labelled Y axis. **A chart without these earns very few marks** — no matter how pretty the bars look.



10

Sections 8 – 10 — Write Up

WRITTEN FINDINGS &
EXECUTIVE SUMMARY · 15
MARKS

The last three sections are entirely written. No new analysis, just synthesis — turning everything above into something a hospital manager would find useful.

8

WRITTEN INSIGHTS

QUESTION Q19

What are the top 5 insights from your analysis?

Write five insights. Each must point to specific evidence from your tables or charts — a number, a ranking, a trend.

X WEAK INSIGHT

"The data is interesting."

✓ STRONG INSIGHT

"Cancer patients are billed the most, averaging about R 28,400 per admission — roughly 18% above the dataset average."

9

EXECUTIVE SUMMARY

Write roughly 150 – 200 words that a busy hospital manager could read in one minute. Cover what the dataset is, how clean it was, and the two or three most important things you found.

★ RULES FOR THE SUMMARY

No code. No jargon. No bullet points. Plain prose. If your manager could not understand the summary, you have not written it correctly. This is the single hardest paragraph in the assignment — leave time for it.



10

RECOMMENDATIONS

QUESTION Q20

What recommendations can you make from your findings?

Give three or four concrete, actionable recommendations. Each one must be tied back to an insight — what should the hospital do differently because of what the data shows?

★ EXAMPLES OF GOOD RECOMMENDATION CATEGORIES

- **Resource planning** — staffing, beds, equipment based on volumes.
- **Data-collection improvements** — what should be captured more reliably next time?
- **Cost reviews** — where do bills look anomalous, and why?
- **Follow-up investigations** — what would you check next if you had more time?



11

Marking Rubric

This is exactly how your notebook will be marked. Read it before you start — and again before you submit.

Criteria	What earns the marks	Marks
Data loading & first inspection	Loads cleanly; shape, dtypes, and head/describe shown	10
Data-quality checks	Missing values, duplicates, and unusual values all investigated	15
Data cleaning & date conversion	Duplicates handled, dates converted to datetime, bad values justified	15
Feature engineering using .loc	All required columns correct; .loc used for the band columns	15
Groupby analysis & sorting	Questions answered with grouped, sorted tables	20
Visualisations	At least 5 clear, labelled charts that support the analysis	10
Written findings & executive summary	Evidence-based insights, summary, and recommendations	15
TOTAL		100

★ WHERE MARKS ARE EASIEST TO LOSE

Section 6 is the largest single section (20 marks) — under-answering one or two questions there costs you more than the same effort spent anywhere else. Section 4 (15 marks) is lost when students convert dates but forget to justify how they handled negative bills. Section 8 (part of the 15-mark write-up) is lost when insights are written without numbers. Spend time where the marks are.



12

Submission Checklist

Before you upload, confirm every line below. This list takes five minutes to walk through and saves the easy marks that the highest-scoring students never lose.

- ☐ Kernel → Restart & Run All completes with no errors.
- ☐ Q1 – Q20 are all answered — code output and written answers where asked.
- ☐ The data dictionary (Section 2) is filled in for all 15 columns.
- ☐ All eight required columns exist — the band columns use .loc.
- ☐ There are at least 5 charts, each with a title and axis labels.
- ☐ Insights, executive summary, and recommendations are in plain English.
- ☐ File renamed to Healthcare_EDA_<YourName>_<StudentNumber>.ipynb.

★ THE FINAL THOUGHT

Remember: the marks are in the explanation, not just the code. A notebook with sloppy code but careful, evidence-based commentary will outscore one with neat code and no interpretation, every single time. Take your time on the write-up. Good luck.



Real data, real questions.

This assignment is your end-to-end exploratory data analysis.
Clean the data, ask good questions, and write the kind of report
a hospital manager could read in one minute.

★ BRIGHTLEARN • HEALTHCARE EDA ASSIGNMENT ★